

From Vibe Coding to Agentic Harness Engineering (agy & claude)

Two-Phase Intensive Engineering Course | Author: Alejandro Kaspar - AI Forward Deployed Engineer (CC BY 4.0 / Apache 2.0)

Phase 1 (Modules 1–4): Vibe Coding Foundations & Workflows

Mastering Daily Execution in Google Antigravity (agy) & Claude Code (claude):

- **Module 1 (Lab 01):** Language-to-code foundations, the 2021–2026 evolution, the 4-Pillar AI Engineering Skills Map, and the Ladder of Earned Autonomy (`prompt_steerer.py`).
- **Module 2 (Lab 02):** Inside the 3-Layer Agent Stack, `agy` 101, `claude` 101, Swiss Army Knife tool primitives, and Wireshark-style harness trace inspection (`wire_trace_inspector.py`).
- **Module 3 (Lab 03):** The 5-step `RePPIT` loop (`Research` → `Propose` → `Plan` → `Implement` → `Verify`), orthogonal proposals + context reset, and reflection (`reppit_orchestrator.py`).
- **Module 4 (Lab 04):** Stateless context window physics, the 40% “Dumb Zone”, output-redirection backpressure, and Frequent Intentional Compaction (`context_compactor.py`).

Phase 2 (Modules 5–8): Harness Engineering & Agentic Loops

Architecting Agent = Model + Harness for Production Codebases:

- **Module 5 (Lab 05):** Cybernetic 2x2 Control Matrix, the 150-Instruction Cliff, the 60-line `CLAUDE.md/GEMINI.md` map rule, and cross-session auto-memory (`context_compiler.py`).
- **Module 6 (Lab 06):** `agentskills.io` 3-level progressive disclosure, `context-hub` (`chub`) self-annotating docs, `SkillsBench` (+16.2pp), and Meta-MCP “Code Mode” (>90% savings).
- **Module 7 (Lab 07):** Subagents as context firewalls, 7-phase `feature-dev`, multi-agent PR triage (`Must Fix` / `Recommended` / `Consider`), and `llm-council` (`subagent_council.py`).
- **Module 8 (Lab 08):** 4-Tier governance, durable replay (`Temporal/Omnigent`), remediation linters, `autoresearch` ratchets, and Recursive Self-Improvement (`autoresearch_harness.py`).

1-to-1 Dual-Branch Parity (agy & claude) + 8 Self-Diagnosing Python Labs (`./self_diagnose_all.sh` in < 1s)

- **Google Antigravity (agy) Branch:** `GEMINI.md` / `AGENTS.md`, `.agents/rules/`, `.agents/skills/`, `.agents/agents/`, `.agents/workflows/`, `.agents/plugins/`, and `KNOWLEDGE.md`.
- **Anthropic Claude Code (claude) Branch:** `CLAUDE.md`, `.claude/rules/`, `.claude/skills/`, `.claude/agents/`, `.claude-plugin/plugin.json`, and `MEMORY.md`.
- **Complete Course Package:** Student Textbook & Reference Manual PDF, Lecturer & Professor Manual PDF, 42-Slide Deck, and Narrated Lecture Videos.

Why Language Produces Code & Actions: Fowler, Wittgenstein & Ng

Unmesh Joshi & Martin Fowler (*What Is Code?*), Wittgenstein's Two Theories of Language, and Andrew Ng's SE Steering Rule

1. Unmesh Joshi & Martin Fowler

What Is Code?:

- Code is not merely syntax for a machine; it is an explicit **externalized conceptual model**.
- Passive “vibe reviewing” creates massive **Cognitive Debt** (synonym drift, untyped dicts, silent assumptions).
- As syntax becomes cheap, **discovering precise domain vocabulary** and invariants becomes the core engineering bottleneck.

2. Wittgenstein's Two Theories

Why Natural Language Coordinates Code:

- **Early *Tractatus* (Picture Theory)**: Formal 1-to-1 logical structure—analogous to compilers, static types, and deterministic unit tests.
- **Late *Investigations* (Language-as-Action)**: Words are active tools in a shared workspace.
- Collaborative prompting aligns intent so the agent emits formal, test-verified code.

3. Andrew Ng's SE Steering Rule

Why Fundamentals Beat Raw Vibes:

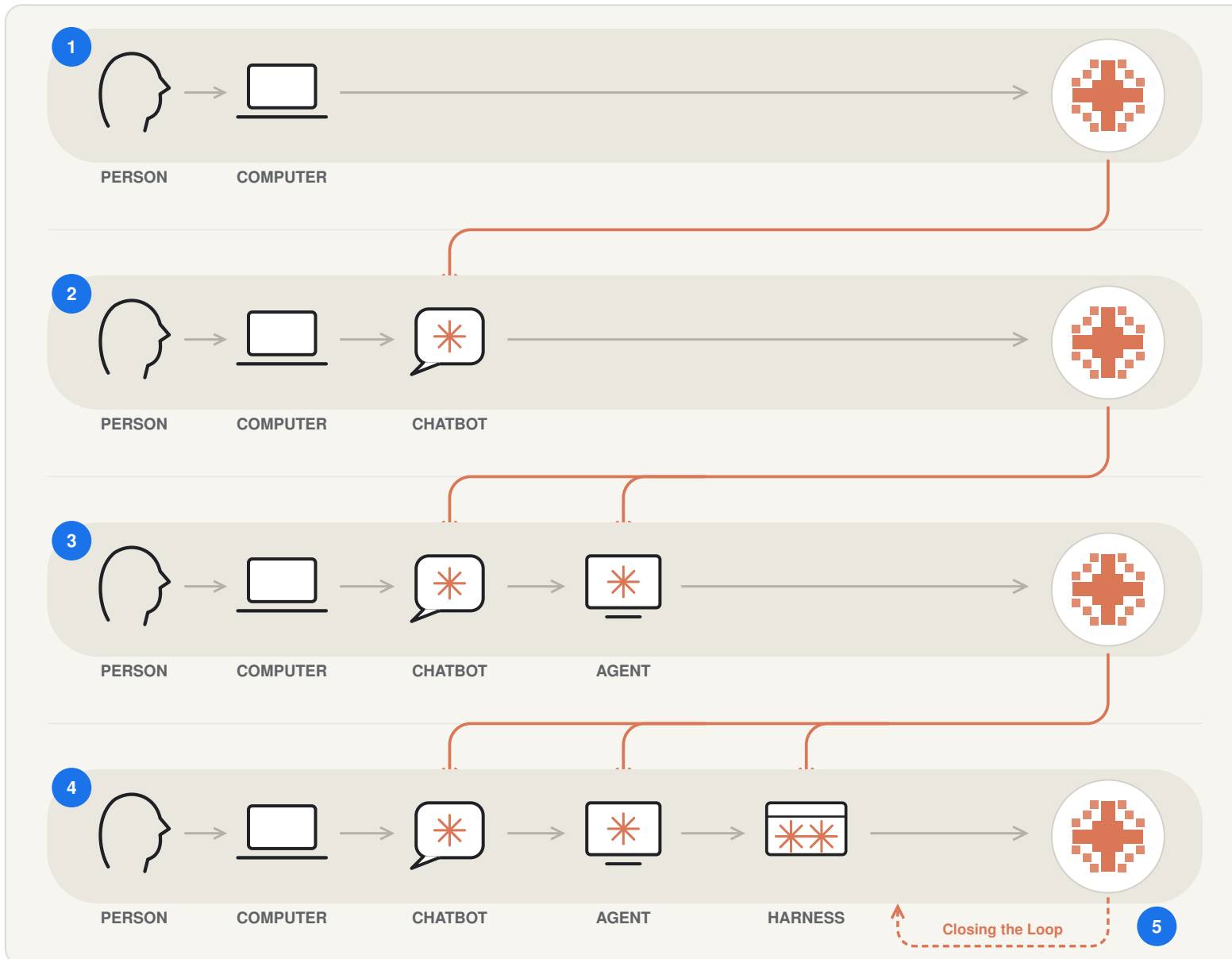
- A novice who vibe codes without **Software Engineering Fundamentals** lets the agent make poor default tradeoffs.
- Without explicit constraints, agents ignore **latency, consistency, idempotency, reliability, maintainability, and blast radius**.
- Precise architectural vocabulary is the steering wheel of an AI coding agent.

The Conceptual Synthesis: Conversational Alignment → Architectural Constraints → Deterministic Verification

- **Naive Vibe Prompt**: "Build a webhook payment handler" → Produces non-idempotent float math, unbounded retries, and SQL injection vectors.
- **SE-Steered Prompt**: "Implement an idempotent webhook handler using integer minor units, p99 < 50ms bounded retries, and strict schema validation."

How Human–AI Software Engineering Evolved: Manual Coding to Closing the Loop

Visualizing how each generation of AI models feeds back into the tools that build the next generation



1. 2021–2023

Building the first models

In the early days, AI development looked like traditional software engineering: people writing code and documentation manually on laptops (assisted by line-level `Tab` autocomplete).

2. 2023–2024

Chatbots & Tool Protocols

Engineers used early chatbots to generate short code snippets and copy-paste output into text editors, while `Function Calling` and `MCP` standardized how models invoke external tools.

3. 2025

Coding agents & Vibe Coding

As models gained RLVR reasoning, coding agents (`claude` & `agy`) became capable of reading repositories, editing entire files, and running tests directly under `CLAUDE.md`, `GEMINI.md`, and `SKILL.md`.

4. TODAY (2026)

Autonomous agents & Harnesses

Agents now run code themselves and delegate hours of work to subagents inside governed harnesses (`git worktree`, `jj`, `Temporal` durable replay, and `Omnigent` meta-harnesses).

5. 2026+

Closing the loop

In bounded verification loops (`autoresearch`, kernel optimization), agents run overnight experiments and evaluate next-step hypotheses to accelerate the next generation of models.

Six Eras of Agentic Infrastructure: Protocols, Context & Harnesses

Key engineering primitives unlocked across each era from early autocomplete to production meta-harnesses

1. Late 2022: Autocomplete & Chat

- **GitHub Copilot GA** (Jun '22): Line-level ghost-text `Tab` completion inside the IDE.
- **ReAct Loop** (Oct '22): Formalizes interleaved `Thought` → `Action` → `Observation` reasoning.
- **ChatGPT Launch** (Nov '22): Conversational code generation via manual copy-paste.

2. 2023: Function Calling & AI IDEs

- **Cursor Launch** (Mar '23): First AI-first IDE with multi-line diff previews.
- **OpenAI Function Calling** (Jun '23): Reliable structured JSON tool invocation.
- **Parallel `tools` API & SWE-bench** (Oct–Nov '23): Multi-tool execution and repository benchmarks.

3. 2024: Long Context, RL & MCP

- **LangGraph & 1M Context** (Jan–Feb '24): Cyclic state machines and repository-scale windows.
- **Project Rules (`.cursorrules`) & OpenAI `o1`** (Jun–Sep '24): Test-time RL reasoning chains.
- **Model Context Protocol (MCP)** (Nov '24): Universal open JSON-RPC bus for tools and resources.

4. Early 2025: Vibe Coding & CLI Agents

- **RLVR & Vibe Coding** (Jan–Feb '25): Verifiable-reward reasoning + Karpathy's Vibe Coding paradigm.
- **Claude Code + `CLAUDE.md`** (Feb '25) & **Gemini CLI + `GEMINI.md` / `AGENTS.md`** (Jun '25).
- **Temporal + Agents SDK & Google A2A Protocol** (Mar–Apr '25): Durable execution and peer agent RPC.

5. Late 2025: Subagents, Hooks & Skills

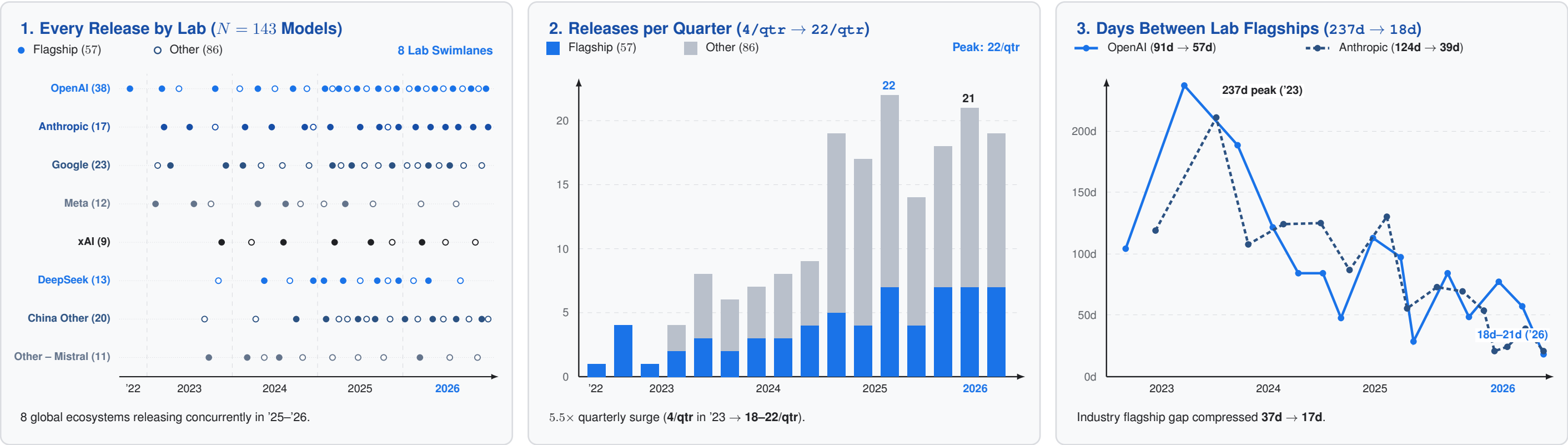
- **Subagents & Lifecycle Hooks** (Jul–Sep '25): Context-isolated child workers and deterministic `PreToolUse` gates.
- **`SKILL.md` & `agentskills.io`** (Oct–Dec '25): Progressive-disclosure procedural manuals.
- **Google Antigravity** (Nov '25): Agent-first IDE & Mission Control with Artifacts and `/browser`.

6. 2026: Harness Engineering & Loops

- **Harness Engineering**: Formalizing Agent = Model + Harness and the 40% “Dumb Zone”.
- **Colocated Jujutsu (`jj`) & Omnigent Meta-Harness**: Conflict-free parallel VCS and multi-agent control planes.
- **Bounded Autoresearch & RSI**: Overnight keep/revert ratchets (`program.md`) and kernel optimization loops.

Why Harnesses Must Decouple from Model Weights: The 17-Day Flagship Pace

Empirical release cadence across 143 frontier models and 8 global lab groups (2022–2026)



Architectural Implication: Build Model-Agnostic Harnesses and Prune Obsolete Scaffolding

- When a new frontier flagship model ships every **17 days** across 8 lab groups, hardcoding brittle prompt workarounds for one model snapshot guarantees technical debt within a month.
- Keep your rules (`CLAUDE.md` / `GEMINI.md`), progressive skills (`SKILL.md`), and verification sensors decoupled from model weights—and re-benchmark on each upgrade to prune obsolete scaffolding.

Andrew Ng’s 4-Pillar AI Engineering Skills Map & Earned Autonomy

Mapping DeepLearning.AI’s 20 skills across 4 Pillars + Karpathy’s Vibe Coding Spectrum & openworker Earned Autonomy

Pillar 01: Building AI Apps

- **1.1 LLM Foundations:** Tokenization, context windows, cache hits, tool calling.
- **1.2 Grounding:** Prompt vs. on-demand retrieval ([chub](#)).
- **1.3 Agentic Systems:** Harnesses, MCP, memory, guardrails.
- **1.4–1.6 Evals & Prod:** Deterministic + LLM-judge evals.

Pillar 02: SE Fundamentals

- **2.1–2.2 Full-Stack & Data:** Schemas, transactions, state.
- **2.3 System Architecture:** Latency, consistency, modularity.
- **2.4 Security & Reliability:** Shift-left scanning, blast radius.
- **2.5 Prod Ops:** Observability & resilience steering.

Pillar 03: Coding Agents

- **3.1 Directing Workflow:** Upfront planning vs. fast execution.
- **3.2 Enabling Autonomy:** Permission tiers & subagents.
- **3.3 Reviewing Work:** Diffs, browser receipts, evals.
- **3.4–3.5 Customizing Harness:** [CLAUDE.md](#)/[GEMINI.md](#), Skills, Hooks.

Pillar 04: Shaping the Build

- **4.1 Driving the Build Loop:** Rapid Idea → MVP → Prod in small batches.
- **4.2 Product Decisions:** Scoping vertical slices.
- **4.3–4.4 High Agency:** “The Conductor Mindset”—orchestrating over abdicating.

Karpathy’s Vibe Coding Spectrum & andrewyng/openworker’s Ladder of Earned Autonomy

1. **Tier 0 – Pure Weekend Vibe Coding** ([MenuGen](#) / [llm-council](#) **hack**): Accept all diffs, ignore code → *Only for throwaway prototypes*.
2. **Tier 1 – Approval-Gated Steering** ([agy request-review](#) / [claude default](#)): Human reviews every edit and shell command.
3. **Tier 2 – Sandboxed Auto-Edit** ([agy proceed-in-sandbox](#) / [claude acceptEdits](#)): Auto-approves local file edits; gates network/shell.
4. **Tier 3 – Bounded Autonomous Loop** ([agy always-proceed](#) / [claude auto](#)): Reviewer-gated with **Hard Floors** (never self-approves destructive ops).

Lab 01: Architecture-Guided Steering & Earned Autonomy (prompt_steerer.py)

Walking through labs/lab_01_vibe_coding_and_earned_autonomy/ across agy and claude

Step 1 & 2: SE Prompt Scorer & Autonomy Tier Router

- **Auditing Naive vs. SE-Steered Prompts** (`score_prompt_steering`):

1. Flags underspecified “pure vibe” prompts lacking architectural constraints.
2. Scores explicit **Pillar 02 SE dimensions**: `latency_sla`, `consistency_model`, `failure_handling`, `security_boundary`, and `verification_command`.

- **Dual-Harness Permission Mapping** (`resolve_autonomy_tier`):

Maps risk profiles to `~/.gemini/antigravity-cli/settings.json` (`request-review`, `proceed-in-sandbox`, `always-proceed`, `strict`) and `.claude/settings.json` (`default`, `acceptEdits`, `plan`, `auto`).

Step 3 & 4: Hard Floors & Circuit-Breaker Verification

- **openworker Hard Floor Invariant**: Even in `always-proceed` or `auto` mode, destructive commands (`rm -rf`, `git push -force`, prod DB drops, secret exfiltration) hit an unbypassable `HARD_FLOOR_BLOCK`.

- **Earned Autonomy Graduation**: Repeated verified runs promote commands from one-off approval → standing allowlist rule; consecutive reviewer denials trip the **circuit breaker** back to `plan` / `request-review`.

- **Run Self-Diagnosis in < 0.05s**:

```
./labs/lab_01_vibe_coding_and_earned_autonomy/self_diagnose.sh
```

Lab 01 Key Takeaway: Autonomy Is Earned by Harness Boundaries, Never Granted Blindly

- Combine **Pillar 02 Software Engineering Vocabulary** in your prompt with **Pillar 03 Permission Tiers & Hard Floors** in your harness settings before letting an agent execute multi-file changes.

Anatomy of a Coding Agent on the Wire (aisuite & Stanford CS146S Lecture 2)

The 3-Layer Agent Stack, 200-line minimal agent vs. production JSONL wire trace, and “Swiss Army Knife” primitives

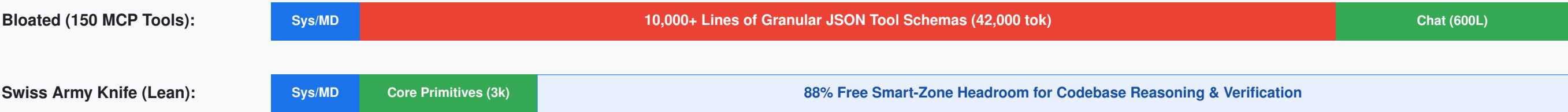
The 3-Layer Agent Stack (andrewyng/aisuite) & Wire Trace

- Layer 1:** Chat Completions API (Stateless `messages[]` → `provider:model`).
- Layer 2:** Agents API · Toolkits (`files`, `git`, `shell`) · MCP.
- Layer 3:** Agent Harness (Context compiler, permissions, hooks, compaction).
- 200-Line Minimal Agent vs. Production HTTP/JSONL Payload:**
- *Minimal Loop:* `while tool_calls: run(read|write|edit|bash).`
 - *Prod Wire Trace:* `System Prompt` + `CLAUDE.md/GEMINI.md` + `Rules` + `Skill YAML Catalog` + `Native Tool Schemas` + `mcp__* Schemas` + `Message History`.

Why “Swiss Army Knife” Primitives Beat 150 CRUD Tools

- CS146S Lecture 2 Wire-Trace Discovery:**
- Mounting a naive 150-endpoint GitHub/Jira MCP server injects **10,000+ lines of JSON schema definitions** for just **600 lines of conversation**!
- The “Swiss Army Knife” Consolidation Principle:**
- Production harnesses rely on **5–8 composable primitives**: `Bash`, `Read`, `Write`, `Edit`, `Grep`, `Glob`, and `LSP`.
 - Frontier models have deep post-trained fluency in `git`, `gh`, `jq`, `ripgrep`, and `pytest` via `Bash`—saving > 85% of schema tokens!

Wire-Trace Token Breakdown: Bloated 150-Tool Harness vs. Lean “Swiss Army Knife” Harness



Google Antigravity (agy) 101: Mission Control, IDE, Artifacts & CLI

Grounded in `getting-started-google-antigravity`, `antigravity-cli-hands-on` & `build-and-deploy-gcp-with-antigravity`

1. Mission Control & IDE

Antigravity 2.0 Desktop & Editor:

- **Agent Manager (Mission Control):** Spawn parallel workspace agents across multi-folder Projects.
- **In-Editor Shortcuts:** `Cmd+L` (AI chat pane), `Tab to Jump` (next logical edit point), and inline `Explain and fix`.
- **/browser Chrome Subagent:** Live DOM inspection, console log capture, and visual verification screenshots.

2. First-Class Artifacts

Verifiable Deliverables over Raw Chat:

- **Task List:** Live structured progress checklist updated autonomously.
- **Implementation Plan:** Blueprint supporting **Google Docs-style inline comments** before clicking **Proceed**.
- **Walkthrough (Run Receipt):** Post-implementation proof bundling diffs, test logs, and browser recordings.

3. Antigravity CLI (agy)

Terminal & Headless Automation:

- **Core CLI:** `agy -version`, `agy models`, `agy -p "..."` `-model ...`, and `!` Shell Mode toggle.
- **Settings:** `~/.gemini/antigravity-cli/settings.json` & `/config`.
- **Power Slash Commands:** `/plan`, `/artifact`, `/grill-me`, `/goal`, `/schedule`, `/stats`, `/memory`, `/skills`, `/compress`, `/clear`.

Antigravity Customization Hierarchy (GEMINI.md + AGENTS.md + .agents/)

- **Workspace Configs:** Root `GEMINI.md` & `AGENTS.md` → `.agents/rules/*.md` → `.agents/skills/<name>/SKILL.md` → `.agents/agents/*.md` → `.agents/workflows/*.md`.
- **Persistent Knowledge Items:** Stored at `~/.gemini/antigravity/knowledge/<project>/KNOWLEDGE.md` and managed via `/memory show|add|refresh`.

Anthropic Claude Code (claude) 101: Surfaces, Modes & Slash Commands

Grounded in Anthropic Academy's `claude-code-101` and DeepLearning.AI's *Claude Code: A Highly Agentic Coding Assistant*

1. Core Loop & Surfaces

Gather Context → Act → Verify:

- Runs natively in **Terminal CLI** (`claude`), **VS Code / JetBrains, Desktop App**, and **Web**.
- **Context Pinning:** Use `@file` and `@dir/` to inject exact source files without noisy grep searches.
- **Direct Bash Mode:** Prefix any command with `!` (e.g., `!pytest -q`) to run shell commands directly in transcript.

2. 3-Stage In-Editor Cycle

Interactive Steering & `Shift+Tab`:

- **`Shift+Tab` Mode Cycling:** Toggle instantly between `Normal` → `Auto-Accept Edits` → `Plan Mode`.
- **Real-Time Interruption:** Queue follow-up messages mid-flight or press `Esc` to halt a runaway tool loop.
- **Headless CI/Script Mode:** Run `claude -p "review diff" -output-format json`.

3. Context & Time-Travel

Essential Session Commands:

- `/init` & `/memory`: Bootstrap and edit `CLAUDE.md` and project `MEMORY.md`.
- `/context`: Inspect live token usage across system, files, MCP, and messages.
- `/compact [focus]` & `/clear`: Compress or wipe history between tasks.
- `/rewind (Esc+Esc)` & `/btw`: Checkpoint rollback and zero-history side queries.

1-to-1 Command Translation Cheat Sheet (`agy` ↔ `claude`)

- **Inspect Context:** `agy /stats` ↔ `claude /context`
- **Compress / Reset:** `agy /compress, /clear` ↔ `claude /compact, /clear`
- **Plan Mode:** `agy /plan` (Implementation Plan Artifact) ↔ `claude Shift+Tab` (`Ctrl+G` editor)
- **Shell Toggle:** `!` in both CLIs.

Lab 02: Wireshark-Style Harness Trace Inspector (`wire_trace_inspector.py`)

Inspecting JSONL wire traces, tool schema bloat, and loop circuit breakers in `labs/lab_02_agent_loop_and_harness_wire_trace/`

Step 1 & 2: Parsing the Wire Trace & Schema Overhead

- **Inspecting Full HTTP/JSONL Payloads (`inspect_wire_trace`):**

1. Dissects a captured agent request into `system_prompt`, `standing_context` (`CLAUDE.md/GEMINI.md`), `native_tools`, `mcp_tools`, and `messages`.
2. Computes exact token shares and flags when `mcp_tools` schema bloat exceeds the **15% schema budget cap**.

- **Consolidating to Swiss Army Knife Primitives (`prune_to_core_primitives`):**

Replaces redundant REST CRUD tool definitions with composable `Bash/Read/Edit/Grep` primitives, recovering $> 80\%$ of fixed startup overhead.

Step 3 & 4: Runaway Loop Detection & Self-Diagnosis

- **Detecting Spin Loops on the Wire (`detect_runaway_tool_loop`):**

1. Identifies repeated identical tool invocations (`tool_name + canonical_args`) or oscillating edit-fail-revert cycles across consecutive turns.
2. Trips a deterministic `CircuitBreakerError` after $N = 3$ repeated failures and emits a structured prompt reset recommendation.

- **Verify Lab 02 in $< 0.05\text{s}$:**

```
./labs/lab_02_agent_loop_and_harness_wire_trace/self_diagnose.sh
```

Lab 02 Key Takeaway: Never Debug an Agent Blind—Inspect the Wire Payload

- Whenever an agent behaves erratically or runs slow, inspect the wire trace (`/stats` or `/context`): 90% of failures stem from hidden schema bloat or polluted message turns.

Task Complexity vs. Workflow Rigor & Defeating “Silent Assumption Syndrome”

Calibrating process overhead to task blast radius (Stanford CS146S Lecture 3) and Socratic Inversion-of-Control via `/grill-me`

Task Complexity vs. Workflow Rigor Spectrum

- **Level 1 – Copy / Typo / Single-Function Tweak:**

Workflow: Direct vanilla prompt with `@file` pin (~1 turn).

- **Level 2 – Small Multi-File Bugfix or Endpoint (2–4 files):**

Workflow: Single **Plan Mode** pass (`agy /plan` or `claude Shift+Tab`) → Execute → Test.

- **Level 3 – Medium Feature or Schema Change (4–12 files):**

Workflow: Full 5-Step **RePPIT Loop** (Research → Propose → Plan → Implement → Verify).

- **Level 4 – Large Cross-Module Refactor / Migration (>12 files):**

Workflow: Multi-session stacked vertical slices with isolated `git worktree` subagents.

Defeating “Silent Assumption Syndrome” via `/grill-me`

- **Why RLHF Models Guess Silently:** Standard LLMs are post-trained to be “helpful immediately,” causing them to silently guess auth schemes, pagination limits, error formats, and retry policies.

- **Socratic Inversion of Control (`/grill-me` & Plan Mode):**

Instead of asking the agent to write code immediately, invert control:

`"Interview me relentlessly about every ambiguity, edge case, and architectural trade-off in this feature. Do NOT write code until all questions are resolved."`

- Surfaces hidden requirements in 2 minutes before a single line of code is written.

Skills Map 3.1 Rule: Match Upfront Planning Depth to Blast Radius

- Under-planning a medium feature causes architectural rewrite loops; over-planning a 3-line CSS fix wastes tokens. Use the 4-level spectrum above on every task.

The 5-Step RePPIT Loop (themodernsoftware.dev CS146S Lecture 3)

Research → Propose (+ Context Reset!) → Plan (+ Out of Scope) → Implement (Model Routing) → Verify & Reflect

1. RESEARCH

Objective Discovery:

- Document *only what exists today* with exact `file:line` citations.
- Strictly **forbidden from proposing solutions** during research.
- Builds both compressed agent context and the **human's mental model**.

2. PROPOSE + RESET

Orthogonal Options:

- Force **2 orthogonal proposals** (e.g., sync DB vs. async queue) with trade-offs.
- **Post-Proposal Context Reset:** Once Proposal B is picked, **purge Proposal A from context** so rejected tokens never confuse coding!

3. PLAN

Locked Blueprint:

- File-and-line edit list in `Implementation Plan` Artifact or `Ctrl+G`.
- Mandatory **Out of Scope / Do NOT Touch** guardrail section blocking scope creep.
- Explicit test verification matrix.

4. IMPLEMENT

Phase Model Routing:

- Use a **frontier reasoning model** for Steps 1–3 (*Research, Propose, Plan*).
- Route Step 4 (*Implement*) to a **fast, cost-efficient coding model** once the design is locked!

5. VERIFY & REFLECT

Critique & Evidence:

- Run `translation-agent Reflection pass` (*Draft → Critique → Refine*).
- Execute unit tests + live `/browser` / Playwright visual evidence in `Walkthrough`.

Two High-Leverage RePPIT Rules That Eliminate 80% of Agent Hallucinations

- **Rule 1 – Post-Proposal Context Reset:** If an agent generates 1,500 tokens describing Rejected Proposal A and 1,500 tokens describing Chosen Proposal B, leaving Proposal A in the chat history causes the model to accidentally blend Proposal A's types into Proposal B's implementation. Always compact/reset after choosing!
- **Rule 2 – Explicit “Out of Scope / Do NOT Touch” Section:** Coding agents love “helpful” drive-by refactors. Listing forbidden files/modules in the plan keeps diffs surgical.

Side-by-Side Daily Execution in agy vs. claude & Time-Travel Recovery

Reviewing plans inline, capturing Run Receipts, and recovering instantly from bad trajectories

Google Antigravity (agy) Execution & Recovery

- **Inline Plan Co-Editing:** Open the generated `Implementation Plan` Artifact, highlight any over-engineered function or missing edge case, and leave a **Google Docs-style inline comment** before clicking **Proceed**.
- **Browser Verification Receipt:** Invoke `/browser` to upload synthetic test data, inspect console errors, and attach screenshots automatically to the `Walkthrough` Artifact.
- **Time-Travel Checkpoint Rollback:** Click `Undo changes up to this point` on any earlier step in the trajectory to revert both workspace files and conversation state simultaneously.

Anthropic Claude Code (claude) Execution & Recovery

- **Interactive Plan Editing (`Ctrl+G`):** Enter Plan Mode via `Shift+Tab`, let Claude draft the plan, and press `Ctrl+G` to open the plan directly in your text editor to trim scope before execution.
- **Playwright MCP Verification:** Connect Playwright MCP to launch headless browser checks and verify rendered DOM state alongside `!pytest`.
- **Time-Travel via `/rewind` (`Esc+Esc`):** Press `Esc` twice (or run `/rewind`) to open the checkpoint menu and restore code only, conversation only, or both.

Why Time-Travel Beats “Arguing in Chat”

- When an agent takes a wrong architectural turn, **never spend 5 turns arguing with it in chat** (which fills the context window with broken code and error logs). Rewind to the plan checkpoint, tighten the constraint in 1 line, and re-run cleanly!

Lab 03: End-to-End RePPIT Orchestrator (reppit_orchestrator.py)

Walking through labs/lab_03_reppit_workflow_and_reflection/ step by step

Step 1, 2 & 3: Research Audit, Proposals & Context Reset

- **Step 1 – Pure Research Validator** (`validate_research_artifact`):

Verifies every claim cites an existing `file:line` and rejects premature code snippets or solution proposals.

- **Step 2 – Orthogonal Proposal & Context Reset** (`select_proposal_and_reset`):

Requires ≥ 2 distinct architectural proposals with explicit trade-off matrices, locks the chosen proposal, and **purges rejected proposal tokens** from the active context state.

- **Step 3 – Plan Gate** (`validate_implementation_plan`):

Enforces non-empty `target_files`, `verification_steps`, and `out_of_scope` guardrails.

Step 4 & 5: Model Routing, Reflection & Run Receipt

- **Step 4 – Phase-Based Model Router** (`route_phase_model`):

Routes `RESEARCH`, `PROPOSE`, and `PLAN` to the frontier reasoning tier, and routes `IMPLEMENT` to the fast execution tier while checking that zero `out_of_scope` files were modified.

- **Step 5 – translation-agent Reflection Pass** (`run_reflection_pass`):

Executes `Draft` $\rightarrow$ `Critique` $\rightarrow$ `Refine` against domain invariants and compiles the verified `Walkthrough` Run Receipt.

- **Verify Lab 03:** `./labs/lab_03_reppit_workflow_and_reflection/self_diagnose.sh`

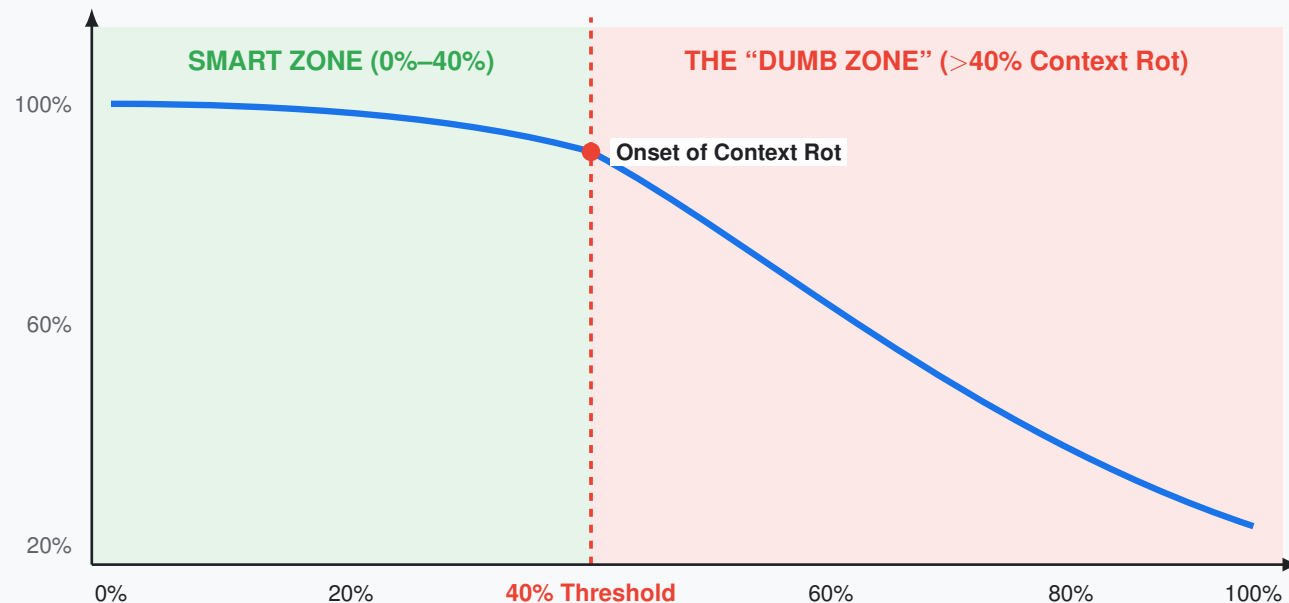
Lab 03 Key Takeaway: Discipline at the Proposal & Plan Gates Makes Implementation Trivial

- By purging rejected proposals, locking `Out of Scope` files, and running a reflection pass before commit, Lab 03 eliminates scope creep and silent regressions.

Context Window Physics (`rmvDxxNubIg`): The 40% “Dumb Zone”

Stateless NextAction = LLM(ContextWindow) physics and why “not all 1M tokens are created equal”

Reasoning & Instruction-Following Quality vs. Context Utilization



Dex Horthy’s First Law of Coding Agents (`rmvDxxNubIg`)

- **Stateless Function Physics:** Every single turn, an LLM computes `NextAction` = `LLM(ContextWindow)` from scratch. The only lever you have to improve the next action is the **signal-to-noise ratio of the context window**.

- **What Floods Context Past 40% into the “Dumb Zone”:**

1. Unfiltered `grep` or `find` dumps across `node_modules/` or build logs.
2. Verbose test/training stdout (5,000 lines of progress bars and tracebacks).
3. Multi-turn “That’s still wrong, try again” correction loops.
4. Bloated MCP tool schemas and monolithic 500-line instruction files.

4-Dimension Context Optimization Hierarchy: Correctness > Completeness > Size (<40%) > Trajectory

- 1. No Incorrect Information (worst poison) → 2. No Missing Information → 3. Minimal Noise (<40% capacity) → 4. Clean Forward Trajectory (never argue with a polluted history).

Context Backpressure (karpathy/autoresearch & rendergit) & Side-Queries

Preventing stdout floods via shell redirection, dual-view repo packing, and zero-history `/btw` queries

1. Output-Redirection Backpressure

Karpathy's `autoresearch` Rule:

- Never let a test suite or training job stream raw output into the LLM context window (*"do NOT use tee or let output flood your context"*).

- **Canonical Pattern:**

```
uv run train.py > run.log 2>&1
```

```
grep "^val_bpb:|^peak_vram:" run.log
```

- On crash, read only `tail -n 50 run.log`. Compresses 50,000 log tokens to **40 tokens!**

2. karpathy/rendergit Dual View

Human HTML vs. LLM CXML View:

- When ingesting a small repository or module, `rendergit` provides two tailored representations:
 1. **Human View:** Syntax-highlighted static HTML with navigable directory tree sidebar.
 2. **LLM CXML View:** Clean, token-counted XML document boundaries (`<document path="...">`) stripping binary/lockfile noise.
- Pair with `karpathy/nanochat` minimalism.

3. Sandboxed Side-Queries (`/btw`)

Zero-Footprint Tangents:

- Mid-implementation, you often need to ask a quick question (*"What flag does `pytest` use for Doctests?"*).
- Asking in the main chat appends the tangent to every future turn.
- **Claude Code `/btw <question>`:** Answers in a modal using current context, then **discards the Q&A from history!**
- Monitor budget via `/context` & `/stats`.

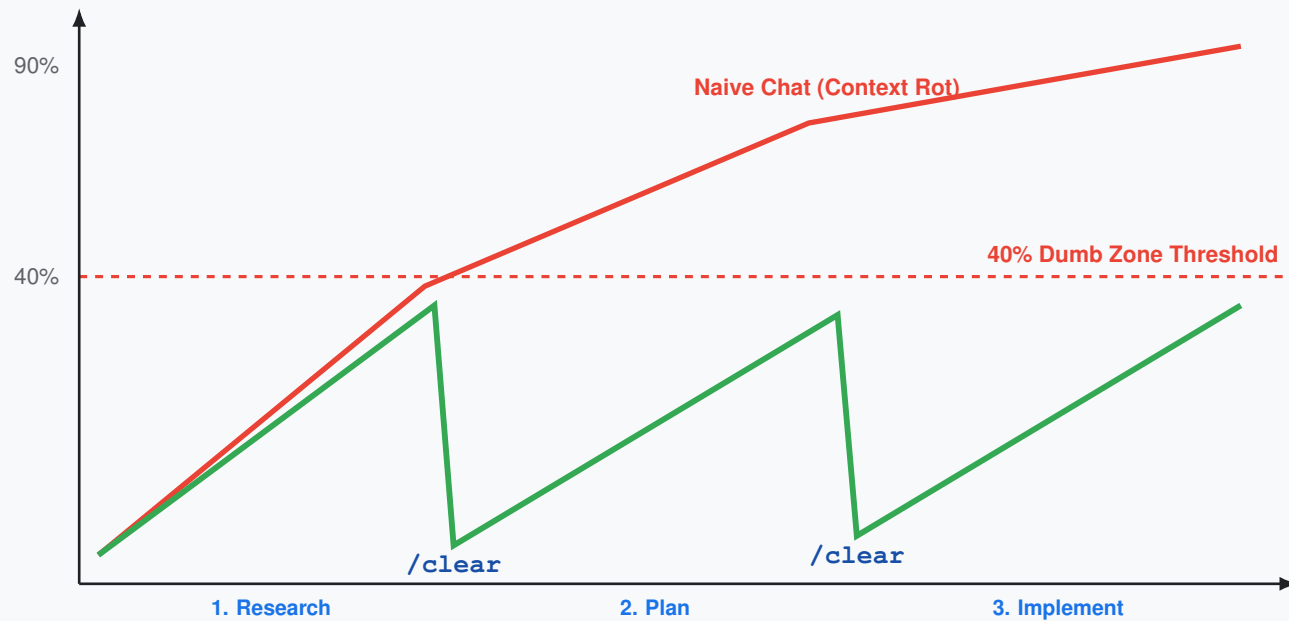
Wrapper Script Pattern for Deterministic Backpressure

- Wrap noisy build/test commands in a helper script that outputs `[PASS] 24 tests (0.12s)` on success and prints *only* the failing assertion traceback on error.

Frequent Intentional Compaction (FIC), RPI & The Human Leverage Pyramid

Sawtooth context management via `research.md` / `plan.md` handoffs and why 1 line of plan review saves 1,000 lines of code

Sawtooth Context Utilization: Naive Accumulation vs. FIC (RPI)



The Human Attention Leverage Pyramid (rmvDxxNubIg)



How Frequent Intentional Compaction (FIC) Works Between RPI Phases

- At the end of **Research**, write verified findings + exact `file:line` citations to `research.md` and run `/clear`.
- Start **Plan** with a fresh window reading only `research.md` → write `plan.md` → run `/clear` → execute **Implement** in the Smart Zone (< 25% utilization)!

Lab 04: Context Budget Profiler & RPI Compaction Engine (context_compactor.py)

Walking through labs/lab_04_context_physics_and_rpi_compaction/ step by step

Step 1 & 2: 40% Dumb-Zone Profiler & Log Backpressure

- **Step 1 – Context Budget Profiler** (`profile_context_window`):

1. Computes total utilization ratio across system instructions, tool schemas, file reads, and tool outputs against the model's context window.
2. Flags `DUMB_ZONE_WARNING` the moment utilization crosses **0.40 (40%)**.

- **Step 2 – Output-Redirection Backpressure** (`apply_log_backpressure`):

Filters multi-thousand-line test/training stdout logs down to summary metrics (`val_bpb`, `passed/failed` counts) + the final 30 traceback lines on failure.

Step 3 & 4: RPI Artifact Compaction & Self-Diagnosis

- **Step 3 – Intentional Compaction Engine** (`compact_to_rpi_artifact`):

1. Distills a noisy exploration transcript into a structured `research.md` or `plan.md` handoff artifact.
2. Validates that every retained claim preserves its canonical `path/to/file.py:L10-L25` citation and strips failed search dead-ends.
3. Resets active session utilization from **>65% back down to <12%**.

- **Verify Lab 04 in < 0.05s:**

`./labs/lab_04_context_physics_and_rpi_compaction/self_diagnose.sh`

Phase 1 Complete! You Now Command Daily Vibe Coding & Context Hygiene in agy and claude

- Next, in **Phase 2 (Modules 5–8)**, we transition from operating the agent interactively to **Harness Engineering**—designing the rules, skills, MCP code mode, subagent firewalls, and ratchets around the model.

Entering Phase 2: Harness Engineering (Agent = Model + Harness)

Birgitta Böckeler & Kief Morris’s 2x2 Cybernetic Control Matrix (martinfowler.com) and Why Loop vs. How Loop

The 2x2 Cybernetic Control Matrix (Böckeler & Morris, martinfowler.com)

	GUIDES (Feedforward / Before Action)	SENSORS (Feedback / After Action)
COMPUTATIONAL (Deterministic CPU)	1. Deterministic Guides - OS/Container Sandboxes (OpenShell) - Tool allowlists & PreToolUse Exit-2 gates - Codemods & static project templates	2. Deterministic Sensors - Compiler & Static Typecheckers - Unit/Integration Tests & PostToolUse - Remediation-Aware Structural Linters
INFERENTIAL (Semantic LLM)	3. Inferential Guides CLAUDE.md / GEMINI.md (60-line map) - Glob-scoped rules/*.md & SKILL.md - Curated API Grounding (chub)	4. Inferential Sensors - Multi-Agent PR Review Subagents - Karpathy’s llm-council Peer Review translation-agent Reflection Pass

Kief Morris: “Why Loop” vs. “How Loop”

- Human-Owned “Why Loop”:** Defines business outcomes, domain architecture, program design (types/interfaces), and trade-offs.
- Agent-Executed “How Loop”:** Explores files, drafts code, runs tests, and self-repairs inside the harness.
- The “On-the-Loop” Harness Flywheel:**
Whenever a defect escapes the How Loop, **do not merely patch the code line**—upgrade one of the 4 Cybernetic Quadrants so that entire class of failure can never recur!

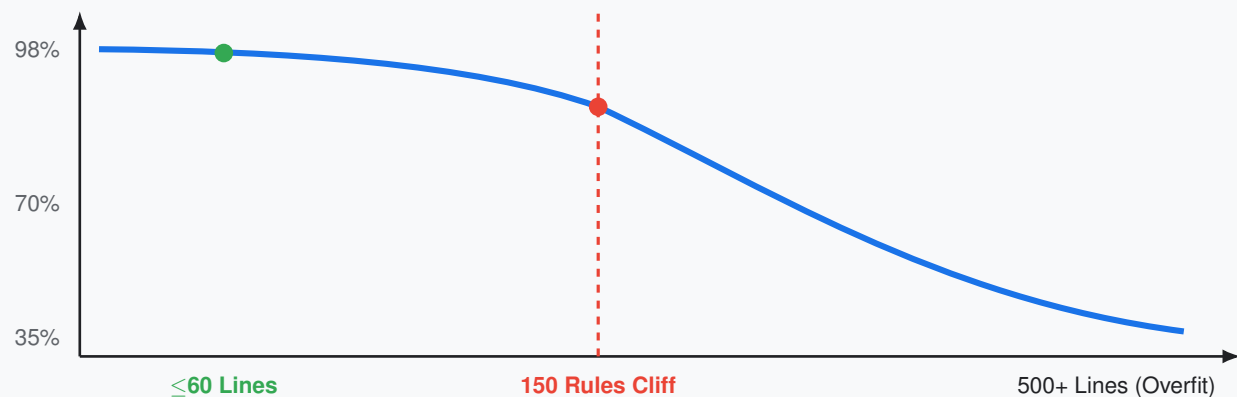
The Golden Rule of Harness Engineering: Prefer Computational (CPU) Checks Over Inferential (LLM) Hopes

- Never write a 20-line prose paragraph in [CLAUDE.md](#) begging the model to format imports or forbid a deprecated API if a 10-line **Deterministic Linter** or [PreToolUse](#) **Hook** can enforce it with 100% certainty at zero token cost!

The Three Context Primitives, The 150-Instruction Cliff & The 60-Line Map Rule

Prompts vs. Rules vs. Skills, why 500-line instruction files overfit (OpenAI & CS146S), and progressive `@docs/` imports

The 150-Instruction Compliance Cliff (Why Monolithic Rules Fail)



The Three Context Primitives:

1. **Prompts / Slash Workflows:** Ephemeral per-turn user instructions (`/plan`, `/grill-me`).
2. **Rules** (`CLAUDE.md/GEMINI.md`, `rules/*.md`): Standing constraints (always-on or glob-scoped).
3. **Skills** (`SKILL.md`): On-demand procedural manuals (`~100 tok` YAML until triggered).

The Minimalist 60-Line Map Rule (OpenAI & CS146S)

- **Why 500-Line Files Hurt:** Over-stuffed instruction files consume instruction budget, contradict stale modules, and **overfit agent exploration**.
- **What Belongs in Root** `CLAUDE.md` / `GEMINI.md` (≤ 60 lines):
 1. **Exact Build / Test / Lint / Server Commands** (e.g., `./self_diagnose_all.sh`).
 2. **Concise Architecture & Directory Map** (top-level package boundaries).
 3. **Progressive** `@docs/*.md` **Pointers** (loaded only when relevant to the task).
 4. **Cross-Tool Symlink:** Symlink `CLAUDE.md` $\leftrightarrow$ `AGENTS.md` (and `GEMINI.md`).
- **Path-Scoped Rules:** Move folder-specific rules into `.claude/rules/api.md` (`paths: ["src/api/**"]`) and `.agents/rules/api.md`.

Context Economy Rule: Treat Root Instruction Lines Like L1 CPU Cache

- If a rule applies to $< 20\%$ of tasks (e.g., database migration conventions or frontend CSS tokens), move it out of root `CLAUDE.md/GEMINI.md` into a path-scoped `rules/*.md` file or an on-demand `SKILL.md`.

Cross-Session Auto-Memory & Retrospectives (MEMORY.md vs. KNOWLEDGE.md)

Persisting durable architectural decisions across sessions while pruning stale assumptions

Claude Code Auto-Memory (MEMORY.md)

- **Storage Path:** `~/.claude/projects/<git-repo>/memory/MEMORY.md` (shared across all `git worktree` checkouts of the repository).
- **Startup Cap:** Only the first **200 lines / 25 KB** of `MEMORY.md` are auto-loaded; detailed sub-notes route to `topics/<slug>.md`.
- **Interactive Inspection:** Run `/memory` to inspect, edit, or delete entries when project architecture changes.

Google Antigravity Knowledge Items (KNOWLEDGE.md)

- **Storage Path:** `~/.gemini/antigravity/knowledge/<project>/KNOWLEDGE.md` with structured topic summaries and artifact pointers.
- **Interactive CLI Commands:** Manage standing knowledge directly via `/memory show`, `/memory add`, and `/memory refresh`.
- **Post-Run Retrospectives (Skills Map 3.4):** After non-trivial sessions, capture discovered gotchas and **update downstream entries** when assumptions change.

Preventing Memory Poisoning: Provenance Tagging & Stale-Assumption Pruning

- Never let an agent record speculative hypotheses or transient debugging state as permanent rules. Require explicit **source provenance** (user instruction vs. commit-verified fact) and prune superseded entries on architecture updates.

Lab 05: Context Compiler & Persistent Memory Architecture (`context_compiler.py`)

Walking through `labs/lab_05_context_primitives_and_memory/` step by step

Step 1 & 2: 60-Line Map Compiler & Path-Scoped Rules

- **Step 1 – Monolith-to-Map Refactorer (`compile_root_context_map`):**
 1. Audits a bloated 400-line instruction file and enforces the strict **60-line root cap**.
 2. Extracts domain-specific sections into `@docs/<topic>.md` lazy imports with path-traversal security checks (`relative_to(project_root)`).
- **Step 2 – Glob-Scoped Rule Matcher (`match_scoped_rules`):**

Loads `.claude/rules/*.md` and `.agents/rules/*.md` only when the active file set matches the rule's frontmatter `paths: globs`.

Step 3 & 4: Worktree Memory Store & Self-Diagnosis

- **Step 3 – Cross-Session Memory & Retrospectives (`MemoryStore`):**
 1. Resolves canonical git-worktree memory paths for both `MEMORY.md` and `KNOWLEDGE.md`.
 2. Enforces the **200-line / 25 KB index cap** (routing overflow into `topics/*.md`) and superseding stale assumptions during post-run retrospectives.
- **Verify Lab 05 in < 0.05s:**

```
./labs/lab_05_context_primitives_and_memory/self_diagnose.sh
```

Lab 05 Key Takeaway: 82% Startup Token Reduction with Zero Loss of Domain Guidance

- Refactoring a monolithic prompt into a 60-line root map + glob-scoped rules + topic-overflow memory keeps baseline context under 4% of the window.

Skills as On-Demand Procedural Manuals (agentskills.io)

3-Level Progressive Disclosure (YAML → SKILL.md → scripts/references) and the 3-Stage Skill Maturity Lifecycle

Level 1: Discovery Catalog

YAML Frontmatter (~100 Tokens):

- Only `name` (lowercase-hyphenated matching folder) and `description` load at session startup.
- Must state both positive triggers (`Use when...`) and negative boundaries (`Don't use for...`).
- 20 installed skills cost just ~2,000 startup tokens!

Level 2: Core Manual

SKILL.md Body (≤ 500 Lines):

- Loaded on-demand only when invoked via `/skill-name` or matched by the semantic router.
- Contains concrete workflow steps, exact CLI recipes, and **experience-grounded gotchas**.
- Links one level deep to Level 3 files.

Level 3: Bundled Resources

scripts/, references/, assets/:

- **scripts/**: Deterministic Python/Bash verifiers executed via shell without reading code into context!
- **references/**: Deep domain tables/schemas read selectively per sub-task.
- Costs **0 tokens** until explicitly accessed.

The 3-Stage Skill Maturity Lifecycle (Skills Map 3.4): Prototype → Team Preview → Production GA (+ Pruning)

- **1. Personal Prototype** (captured from a real debugging trajectory) → **2. Team Preview** (checked into `.agents/skills/` & `.claude/skills/`) → **3. Production GA** (gated by paired `benchmark.json` evals) → **4. Retire/Prune** when newer frontier model weights absorb the capability natively!

Curated API Grounding & Self-Improving Annotations (andrewyng/context-hub)

How @aisuite/chub (get-api-docs/SKILL.md) eliminates stale-API hallucinations and compounds session discoveries

1. Stale Pretraining Problem

- LLMs hallucinate deprecated SDK methods (e.g., old `openai.ChatCompletion.create` or stale Stripe webhook signatures).
- Raw web search pulls noisy HTML ads and mismatched versions.

2. chub Curated Fetch

- Agent triggers `get-api-docs/SKILL.md` and runs:

```
chub search stripe  
chub get stripe/api -lang py
```
- Injects clean, versioned, language-specific Markdown docs on demand.

3. Local Self-Annotation

- When the agent discovers a non-obvious SDK quirk during testing, it persists a local note:

```
chub annotate stripe/api "Requires  
raw bytes payload for HMAC check"
```
- Stored locally for future sessions!

4. Guarded Re-Use & Feedback

- Future sessions run `chub get ... -with-annotations` (wrapped as untrusted context so notes cannot inject commands).
- Emits quality signal upstream via `chub feedback up|down`.

Why andrewyng/context-hub Outperforms Static RAG and Raw Documentation Scraping

- By combining **curated version-pinned Markdown packages** with **local agent-authored annotations** (`chub annotate`), the harness turns every hard-won API debugging discovery into permanent institutional memory across sessions.

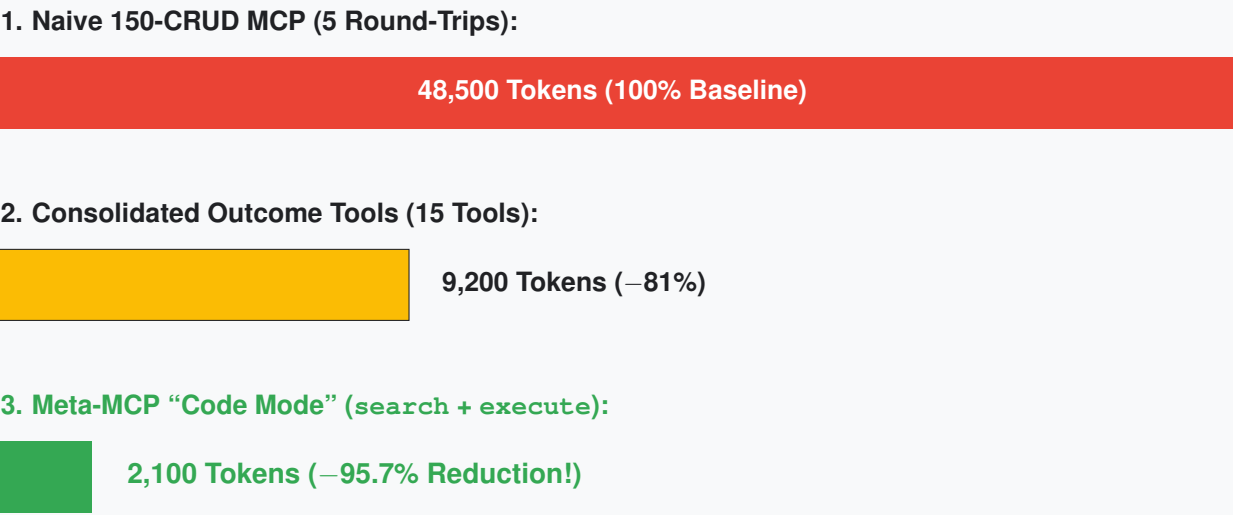
Ergonomic MCP Tool Design & Meta-MCP “Code Mode” (>90% Token Savings)

Why 1-to-1 REST CRUD wrappers fail and how Stanford CS146S Lecture 4’s 4 Rules cut tool tokens by >90%

The 4 Rules of Ergonomic Agent Tools (CS146S Lecture 4)

- 1. Design Around Outcomes, Not Granular REST CRUD:**
Replace 3 chained REST lookups (`get_user` → `get_wallet` → `post_tx`) with 1 outcome tool (`send_payment_to_email`) or consolidated `search_resource(type=...)`.
- 2. Enforce Strict Typed Schemas (Pydantic / TypeScript):**
Ban `*args`, `**kwargs` and untyped `dict` payloads; constrain enums and value bounds.
- 3. Return Actionable, Self-Repairing Error Messages:**
Instead of `HTTP 400 Bad Request`, return exact parameter remediation instructions.
- 4. Expose Meta-MCP “Code Mode” (`search` + `execute`):**
Instead of loading 150 tool schemas upfront, expose only `search(query)` (to fetch 1–2 relevant schemas on demand) and `execute(script)` (running chained calls in a sandbox VM).

Context Tokens Consumed by 5-Step Multi-Tool Workflow



Code Mode executes intermediate pagination and filtering loops inside the sandbox VM so only the final distilled result enters the LLM context window.

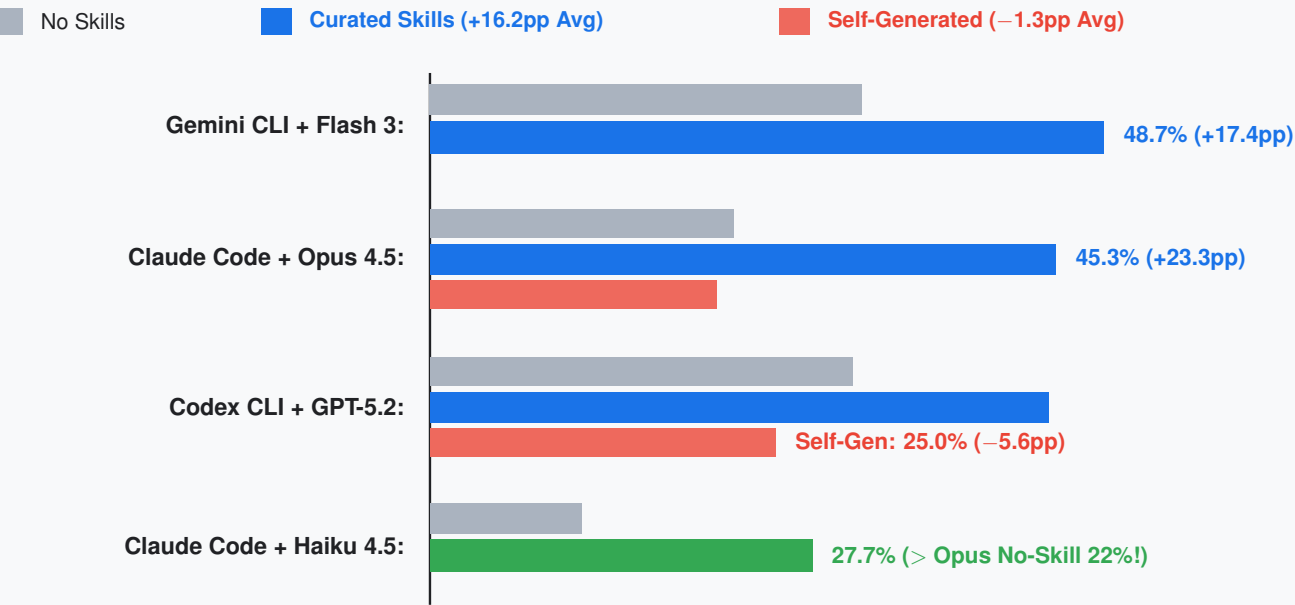
Why Meta-MCP “Code Mode” Is the Architecture Standard for Large Tool Catalogs

- LLMs are far better at writing a 6-line Python/TypeScript script inside `execute()` that loops over paginated API responses than issuing 12 sequential JSON tool-call round-trips across the context window.

Empirical Skill Science (Part 1): Curated vs. Self-Generated Skills & Domains

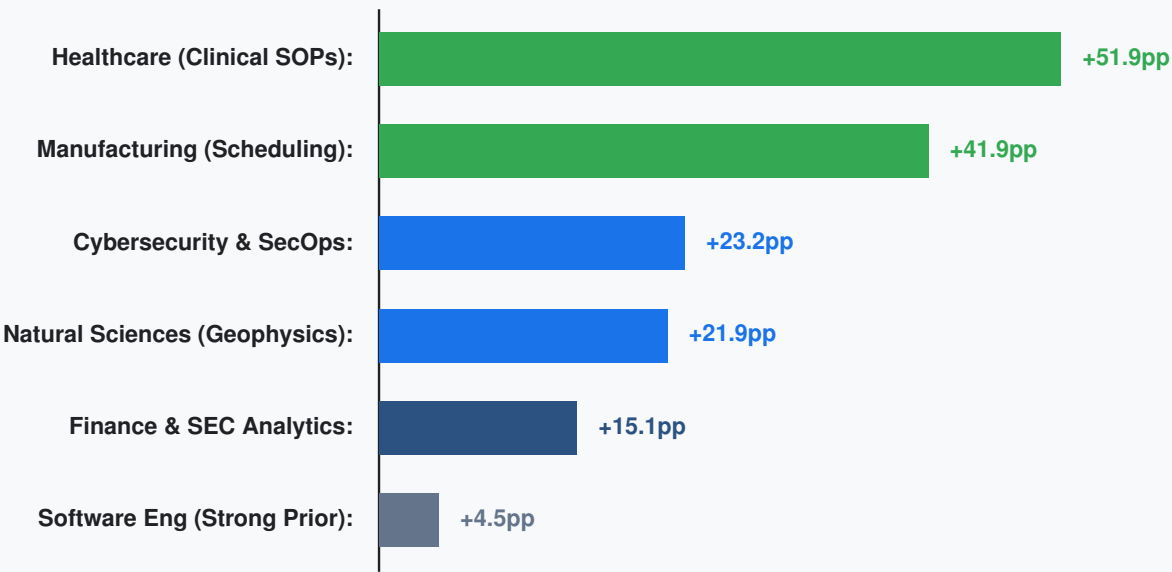
Benchmarking $N = 7,308$ agent trajectories across 84 tasks and 11 domains

1. Pass Rate by Harness & Condition (Curated vs. Self-Gen)



A fast model with curated Skills (27.7%) beats a frontier model without Skills (22.0%).

2. Pass-Rate Gain by Domain (+4.5pp to +51.9pp)



Domains with specialized procedural SOPs see massive gains; generic SWE priors need team-specific deltas.

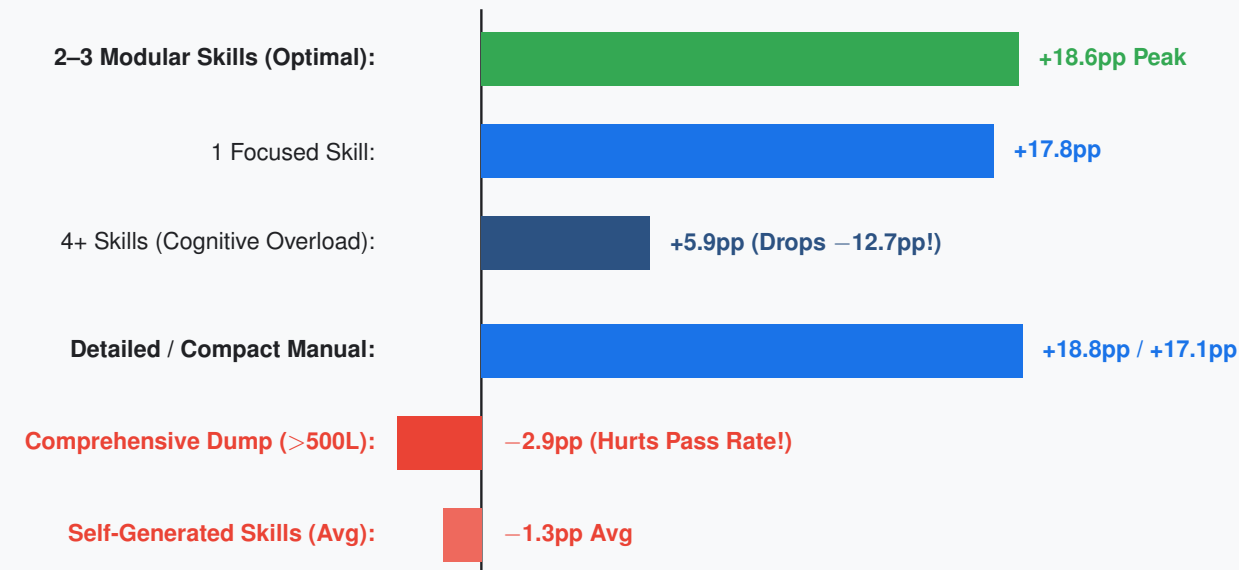
Why Self-Generated Skills Degrade Performance (–1.3pp Avg): Models Cannot Reliably Author Their Own SOPs

- When asked to generate their own skills before solving a task, models produce vague platitudes (“*use pandas for data processing*”) or hallucinate incorrect API flags.
- High-leverage skills must be distilled from **verified human and agent debugging trajectories** and focused on **non-obvious procedural edge cases**.

Empirical Skill Science (Part 2): The 2–3 Skill Sweet Spot & Failure Reduction

Why “less is more” in skill design and how curated skills cut trajectory failures by > 30%

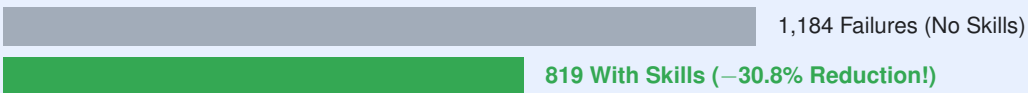
3. Skill Quantity Sweet Spot & Documentation Length



Non-Monotonic Law: Keep 2–3 focused skills (≤ 500 lines); monolithic dumps hurt pass rates.

4. Trajectory Failures Cut by Skills & Cost-Performance Pareto

A. Quality Below Threshold (49.8% of all failures):



B. Incomplete Solution / Premature Stop (Coherence):



- **Cost-Performance Pareto Frontier:** Fast models with curated Skills (`Gemini 3 Flash + Skills` at **48.7%**, **\$0.57/trial**) cut cost by **47%** compared to heavy models while achieving top-tier accuracy.
- **Prompt Caching Multiplier:** Because `SKILL.md` and `CLAUDE.md/GEMINI.md` prefixes remain static across turns, harness prefix caching reaches **63%–99%**.

Mandatory Skill Release Gate: Paired `with_skill` vs. `without_skill` Evaluation (`benchmark.json`)

- Because **16 out of 84 benchmark tasks** experience negative deltas when skills introduce conflicting instructions or unnecessary complexity, always run paired `eval-viewer` benchmarks before promoting a skill to your team repository.

Lab 06: Progressive Skills, chub Annotations & Meta-MCP (`skill_and_mcp_harness.py`)

Walking through `labs/lab_06_skills_chub_and_meta_mcp_code_mode/` step by step

Step 1 & 2: `agentskills.io` Validator & chub Grounding

- **Step 1 – 3-Level Skill Package Validator (`validate_skill_package`):**

Verifies YAML frontmatter (`name`, positive/negative `description` triggers), $\leq$ 500-line `SKILL.md` body, one-level-deep `references/` links, and rejects self-generated filler platitudes.

- **Step 2 – Self-Annotating Doc Store (`ChubDocRegistry`):**

Implements `search()`, versioned `get(id, lang, with_annotations=True)`, local `annotate(id, note)` persistence, and `feedback(id, vote)`.

Step 3 & 4: Meta-MCP “Code Mode” & `benchmark.json`

- **Step 3 – Meta-MCP `search` + `execute` Harness (`MetaMcpCodeMode`):**

1. Exposes only 2 meta-tools (`search_tools` and `execute_code_plan`) over a catalog of 120 backend tools.
2. Proves >90% **schema token reduction** while enforcing strict typed input validation and actionable remediation errors.

- **Step 4 – Paired Ablation Report & Self-Diagnosis:**

`./labs/lab_06_skills_chub_and_meta_mcp_code_mode/self_diagnose.sh`

Lab 06 Key Takeaway: On-Demand Progressive Disclosure Applies to Both Skills and MCP Tools

- Loading skills via Level-1 YAML and loading tools via Meta-MCP `search` + `execute` ensures your agent scales to 100+ capabilities while keeping > 85% of its context window free.

The Interaction Method Decision Matrix & Subagents as Context Firewalls

Choosing between Conversational Prompts, One-Off Prompts, and Subagents

1. Conversational Prompts

When to Use Interactive Turns:

- High-ambiguity requirement discovery (`/grill-me`) and architectural trade-off selection.
- Iterative UI/UX refinement where human judgment steers each step.
- *Warning:* Never use conversational turns for broad 30-file search sweeps!

2. One-Off Bounded Prompts

When to Use Single-Shot / Headless:

- Well-specified transformations with deterministic verification (`agy -p` / `claude -p`).
- Generating commit messages, changelog entries, or single-file unit test suites.
- Zero multi-turn history accumulation.

3. Subagents = Context Firewalls

The Subagent Isolation Law:

- **Never anthropomorphize subagents** into cute role-play personas (*"Frontend Bob"*, *"PM Alice"*).
- Subagents exist for one physical purpose: **Context Isolation**.
- Child burns **50,000 tokens** searching files/logs → returns a **500-token cited summary** to the parent window!

Context Firewall Compression Ratio: 50,000 Child Exploration Tokens → 500 Parent Handoff Tokens (100:1 Savings)

- Because the child subagent's context window is garbage-collected the moment it returns its structured findings, the parent orchestrator stays at < 15% context utilization!

7-Phase feature-dev & Multi-Agent Pull-Request Triage

Orchestrating `code-explorer`, `code-architect`, and 5 specialized `code-reviewer` subagents

Phases 1–3: Explore & Clarify

- **Phase 1 (Discovery):** Frame feature intent.
- **Phase 2 (`code-explorer`):** 2–3 parallel **read-only** sub-agents (`Read`, `Grep`, `Glob` only; `Write/Edit` forbidden) map execution paths with `file:line` citations.
- **Phase 3 (Clarification Gate):** Hard STOP until all open questions are answered.

Phases 4–5: Architect & Build

- **Phase 4 (`code-architect`):** Evaluates Minimal-Change vs. Clean-Architecture vs. Pragmatic lenses; locks file-and-line blueprint + `Out of Scope` list.
- **Phase 5 (Implementation):** Executes surgical edits against the approved blueprint and runs deterministic test suites.

Phases 6–7: 5-Agent PR Triage

- **Phase 6 (`code-reviewer` Fan-Out):** Spawns 5 parallel review subagents: **Security**, **Performance**, **Architecture**, **Style**, and **Edge Cases**.
- Filters out noise below **confidence** ≥ 80 .
- **Phase 7 (Summary):** Synthesizes final Run Receipt.

Multi-Agent PR Triage Taxonomy (Confidence ≥ 80 Gate)

- **[RED] Must Fix (Confidence ≥ 90 , Critical/High):** Security vulnerabilities, data-loss bugs, race conditions, or broken invariants (blocks merge).
- **[YELLOW] Recommended (Confidence ≥ 80 , Medium):** Missing boundary test cases, N+1 query inefficiencies, or error-handling gaps.
- **[GREEN] Consider (Confidence ≥ 80 , Low):** Optional readability simplifications; everything with confidence < 80 is automatically suppressed!

Karpathy's 11m-council: 3-Stage Anonymized Cross-Model Verification

Eliminating single-model self-preference bias via parallel first opinions, blind peer ranking, and chairman synthesis

Stage 1: Independent First Opinions

- Dispatch the architectural prompt or code review diff in parallel to K diverse frontier models.
- Each model formulates its own independent solution or critique in complete isolation without seeing peer drafts.
- Captures orthogonal reasoning priors across model families.

Stage 2: Anonymized Peer Review

- Strip all model names and metadata, labeling candidates strictly as `Response A`, `Response B`, `Response C`.
- Every model critiques and rank-orders the anonymized responses on accuracy, edge-case coverage, and simplicity.
- Prevents self-preference bias and brand favoritism.

Stage 3: Chairman Borda Synthesis

- Aggregate peer rankings using deterministic **Borda Count** scoring ($K - r$ points per rank r) while excluding self-votes.
- The Chairman model synthesizes the top-ranked architecture while incorporating valid edge-case fixes raised by dissenting peers.

Why Cross-Model Peer Review Beats Single-Model Self-Critique

- When a model reviews its own code in the same session, it shares the exact same blind spots that produced the bug. Anonymized cross-model peer review forces independent verification before merge.

Agentic Version Control at Scale: `git worktree` vs. Colocated Jujutsu (`jj`)

Solving Provisioning Cost vs. Integration Cost when parallel subagents edit a shared repository

1. `git worktree` (Cuts Provisioning Cost)

- **Shared Object Store:** Mounts lightweight working directories over a single `.git` directory (`agy` Mission Control & `claude --worktree`).
- **Fast Cold-Start Recipe:** Combine `git clone --filter=blob:none` with a warm reference cache and shared dependency cache (`pnpm`, `uv`).
- **Where `git` Struggles with Swarms:** Rebasing a 4-branch agent stack hits a text conflict and **aborts mid-rebase** (`REBASE 1/4`), freezing the unattended agent loop!

2. Colocated Jujutsu (`jj git init -colocate`)

- **Working Copy = Commit (@) & Stable Change IDs:** Every file edit auto-snapshots without staging ceremony; Change IDs survive rebases.
- **First-Class Conflict Algebra** ($\Delta M = B + C - A$): Rebases **never fail or abort!** Conflicts are stored inside commit objects, descendant commits auto-rebase, and multi-branch megamerges (`jj new A B C`) compose cleanly.
- **Transactional Recovery:** `jj op log` & `jj undo` roll back any multi-branch mistake; `jj absorb` routes fixes to the exact commit in a stack.

The Production Hybrid Pattern: `git worktree` for Fast Provisioning + Colocated `jj` for Conflict-Free Integration

- Use `git worktree` when native Git features (LFS, submodules, `.gitattributes`) dominate; layer colocated `jj workspace add` when parallel agents rewrite stacked PRs and resolve conflicts asynchronously.

Lab 07: Context-Firewall Subagents & Peer Council (subagent_council.py)

Walking through labs/lab_07_subagent_firewalls_and_peer_council/ step by step

Step 1 & 2: Read-Only Firewall & 3-Tier PR Triage

- **Step 1 – Subagent Context Firewall** (`run_isolated_subagent`):

1. Enforces strict tool allowlists (`code-explorer` blocks `Write/Edit/Bash`).
2. Discards the child's multi-thousand-token exploration trace and returns only a bounded ≤ 500 -token cited handoff summary.

- **Step 2 – Multi-Agent PR Triage** (`synthesize_pr_triage`):

Aggregates findings from Security, Performance, Architecture, Style, and Edge-Case reviewers, drops findings with `confidence < 80`, and buckets into `MUST_FIX`, `RECOMMENDED`, and `CONSIDER`.

Step 3 & 4: Anonymized llm-council & Worktree Check

- **Step 3 – Anonymized Borda Peer Council** (`run_anonymized_council`):

1. Scrubs model/agent names into `Candidate_A..N` before cross-review.
2. Computes Borda count scores across peer ballots while blocking self-voting manipulation, and synthesizes the winning design.

- **Step 4 – Parallel Worktree Collision Guard** (`verify_worktree_isolation`):

Blocks concurrent write subagents that share a working tree or have overlapping file globs.

- **Verify Lab 07 in < 0.05s:**

```
./labs/lab_07_subagent_firewalls_and_peer_council/self_diagnose.sh
```

Lab 07 Key Takeaway: Multi-Agent Architecture = Context Isolation + Bias-Free Verification

- Subagent firewalls keep the parent context window clean, while anonymized peer review and confidence-filtered PR triage catch subtle architectural defects before merge.

4-Tier “Governed by Design” Harnesses & Durable Execution Replay

Enforcing deterministic Hard Floors, Earned Autonomy, and crash-proof Event-History Replay over 16-hour horizons

1. Hard Floors (Exit 2)

- “The fixer is never the only checker.”
- Deterministic `PreToolUse` hooks return **Exit Code 2** to block edits to immutable test suites, secrets, or evaluators.
- Unbypassable even in `auto` or `always-proceed` modes.

2. Earned Autonomy

- Secondary reviewer model + static analysis (`semgrep`) inspects proposed tool calls.
- **Stateful Contextual Policies:** Escalates `git push` after `npm install`; triggers human sign-off every **\$100** of API spend.

3. Durable Event Replay

- **Temporal / DBOS / Restate / LangGraph:**
- Separates deterministic `Workflow` orchestration from non-deterministic `Activities` (LLM & MCP calls).
- Reconstructs exact state from the append-only Event Log after crashes or 429s.

4. Egress-Proxy Sandbox

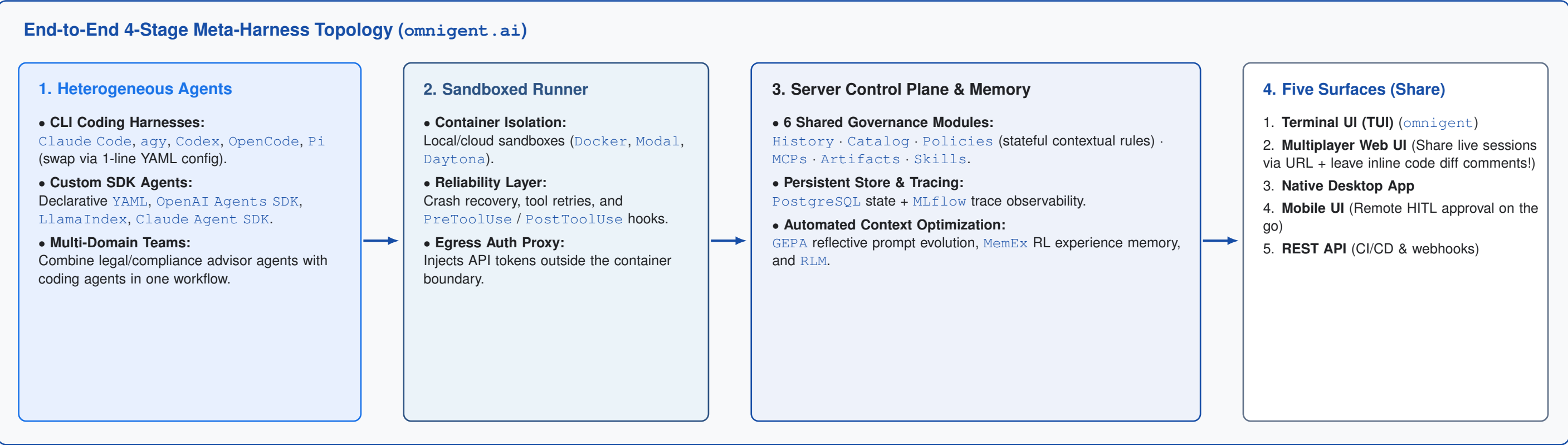
- **OpenShell / Modal / Daytona:**
- Runs agent bash commands inside isolated containers.
- **Egress-Proxy Token Injection:** Injects credentials at the network boundary so the agent never sees raw keys!

Why In-Memory `while` Loops Fail Past 30 Minutes: The Durable Execution Equation

- Over a 12–16 hour autonomous horizon, transient HTTP 503s, container restarts, or 48-hour human approval waits are guaranteed. Checkpointing every LLM turn and tool result as an immutable `Activity` guarantees zero lost tokens and idempotent recovery.

The Enterprise Meta-Harness Pattern (Omnigent): Combine, Control & Share

How a unified Meta-Harness orchestrates heterogeneous CLI & SDK agents across sandboxed runners and multiplayer surfaces



Why Meta-Harnesses Are Emerging Above Individual CLI Agents

- Teams rarely use only a single agent in isolation. A Meta-Harness lets engineers combine specialized harnesses (agy, claude, domain SDK agents), enforce unified egress and spending policies, and collaborate in multiplayer review sessions.

Remediation-Aware Architectural Linters (OpenAI Harness Engineering)

Why standard linter messages fail agents, and how to emit [VIOLATION] → [WHY] → [HOW TO FIX] recipes

Why Traditional Human Linter Messages Fail Agents

- **Cryptic Error Codes:** A standard linter outputs:

```
src/api/orders.py:42: E901 Forbidden import db.session
```

- **What the Coding Agent Does Next:**

1. Does not know *which* module it is supposed to call instead.
2. Wastes 4 tool calls grepping the codebase for workarounds, or worse, adds `# noqa / pylint: disable` to silence the check!
3. Pollutes the context window with trial-and-error guesses.

OpenAI's Remediation-Aware Linter Pattern

- **Write Linter Errors for an LLM Reader ([VIOLATION] → [WHY] → [HOW TO FIX]):**

```
[VIOLATION] Direct import of 'db.session' in 'src/api/orders.py:42' violates Layer Boundary Rule ARCH-03 (API ↗ DB).
```

```
[WHY] Route handlers must remain stateless and testable without a live database connection.
```

```
[HOW TO FIX] Replace 'from db.session import query' with 'from services.order_service import OrderRepository' and inject via 'Depends(get_order_repo)'.
```

- Enables **single-turn autonomous self-repair** on the very next step!

Encoding Architectural Invariants as Fast AST / Regex Structural Checks

- Enforce layer dependency DAGs (`ui → api → service → repo`), ban raw floats in monetary modules, require structured logging, and forbid `# noqa` suppressions via deterministic `PostToolUse` checks.

Karpathy's autoresearch (program.md) & Overnight val_bpb Ratchet

3-File Harness Separation + progress.png: 83 Overnight Experiments, 15 Kept Improvements (0.9979 -> 0.9773)

1. prepare.py (Immutable Sensor)

- **Read-Only Ground Truth:** BPE tokenizer, dataloader, fixed 5-min wall-clock budget, and evaluate_bpb().
- Protected by PreToolUse Exit-Code-2 Hard Floor.

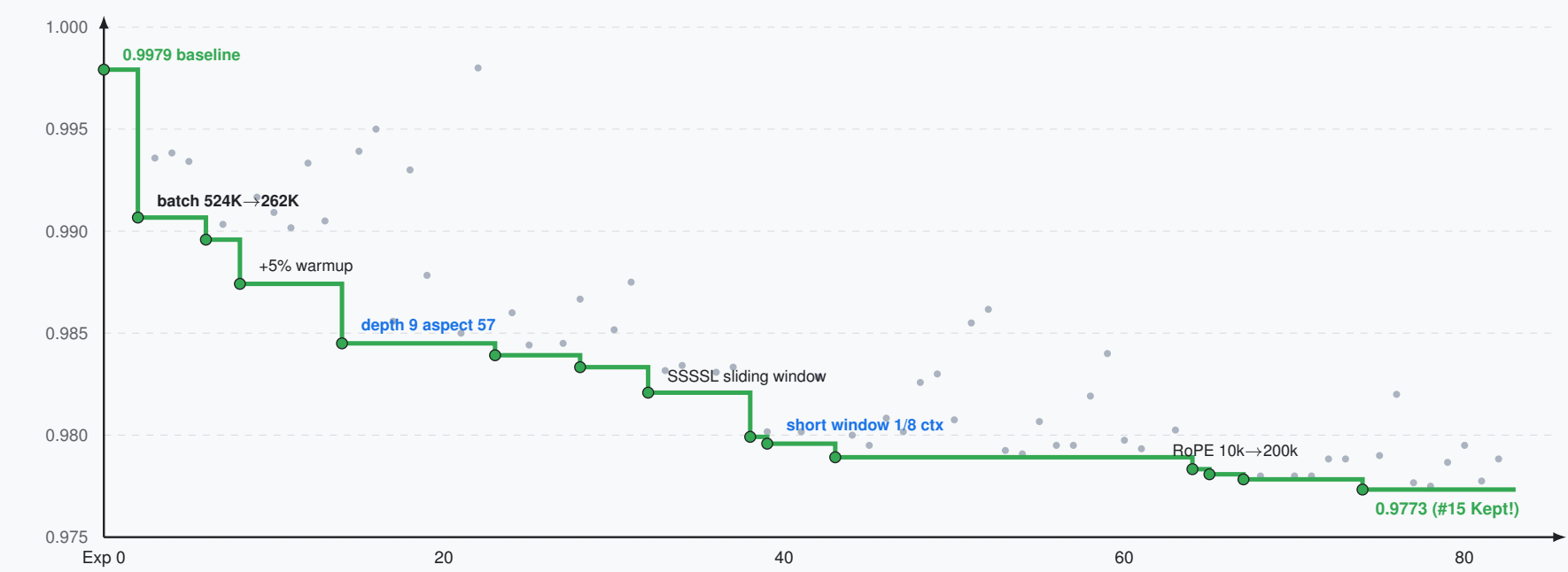
2. train.py (Bounded Action Space)

- **Single Mutable File:** The *only* file the agent edits (GPT architecture, Muon+AdamW optimizer, schedule).
- Zero cross-repo blast radius; every trial is 1 commit.

3. program.md ("Research Org Code")

- **Human-Authored Markdown Spec:** Programs the autonomous research loop, output redirection (> run.log 2>&1), and the **Simplicity Criterion**.

Autoresearch Progress: 83 Exps, 15 Kept (progress.png)



The Keep-or-Revert Ratchet

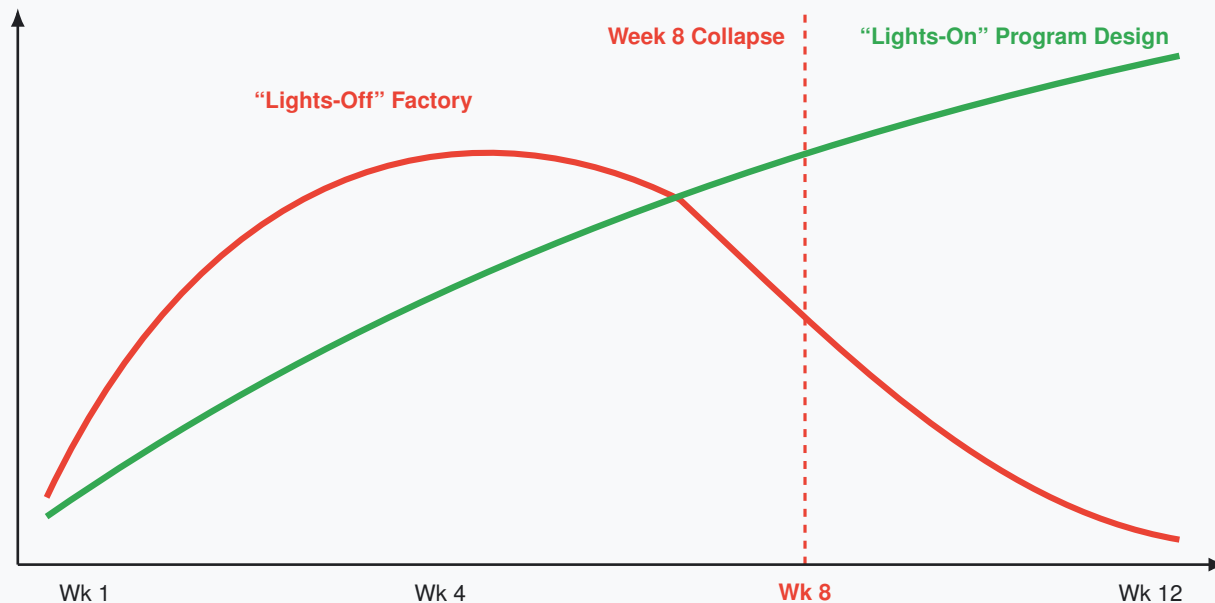
- **Overnight Loop (~12 Exps/Hr):**
 1. Edit train.py & git commit.
 2. Run uv run train.py > run.log 2>&1 (5-min fixed budget; zero stdout pollution).
 3. Extract grep "^val_bpb:" run.log and append to untracked results.tsv.
 4. If improved & simple: **KEEP** commit.
 5. If worse/crash: **git reset -hard HEAD^!**
- **Karpathy's Simplicity Criterion:**

"A 0.001 improvement that adds 20 lines of hacky code? Revert. A 0.001 improvement from deleting code? Definitely keep!"

Why “Lights-Off” Software Factories Fail: The RLVR Horizon Gap

Why optimizing solely for $t = 0$ test pass causes Week-8 architectural collapse, and the 4-Phase Front-Loaded Fix

Feature Velocity & Maintainability Over Time (Weeks 1–12)



The RLVR Reward Horizon Gap

- **Why Dark Factories Collapse at Week 8:** RLVR trains models to maximize $t = 0$ **immediate rewards** (passing current tests, compiling cleanly).
- But software architecture lives on a $t = 6$ **month horizon**! Left unattended in a “lights-off” factory, agents duplicate abstractions, copy-paste 400-line handlers, and create spaghetti coupling that passes $t = 0$ tests until the codebase collapses.
- **Deterministic Harnesses Raise the Floor, Not the Ceiling:**
Linters and tests prevent syntax/layer bugs (floor), but **humans must own the architectural ceiling** via 4-Phase Front-Loaded Alignment.

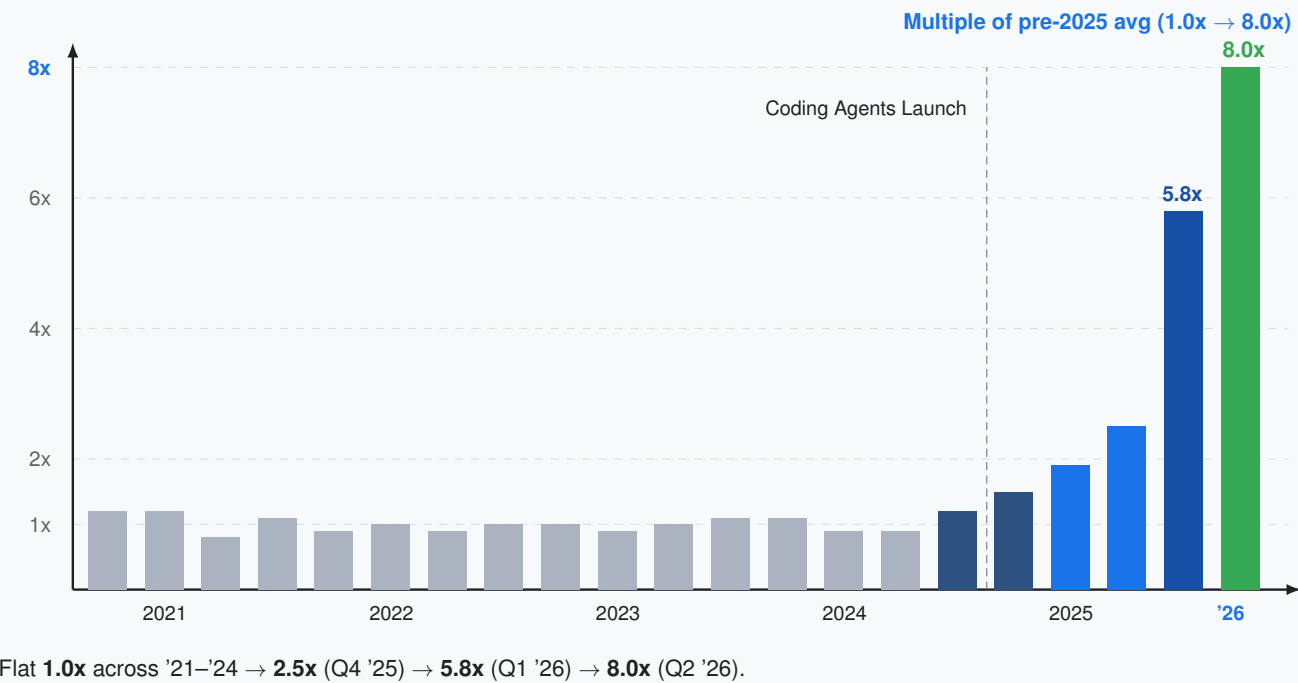
Keeping the “Lights On”: 4-Phase Front-Loaded Alignment

- **Phase 1: Product** (User value) → **Phase 2: Architecture** (System boundaries) → **Phase 3: Program Design** (Exact types, interfaces, file-trees, and call-stacks co-designed with the human!) → **Phase 4: Vertical Slices** (Bounded agent implementation).

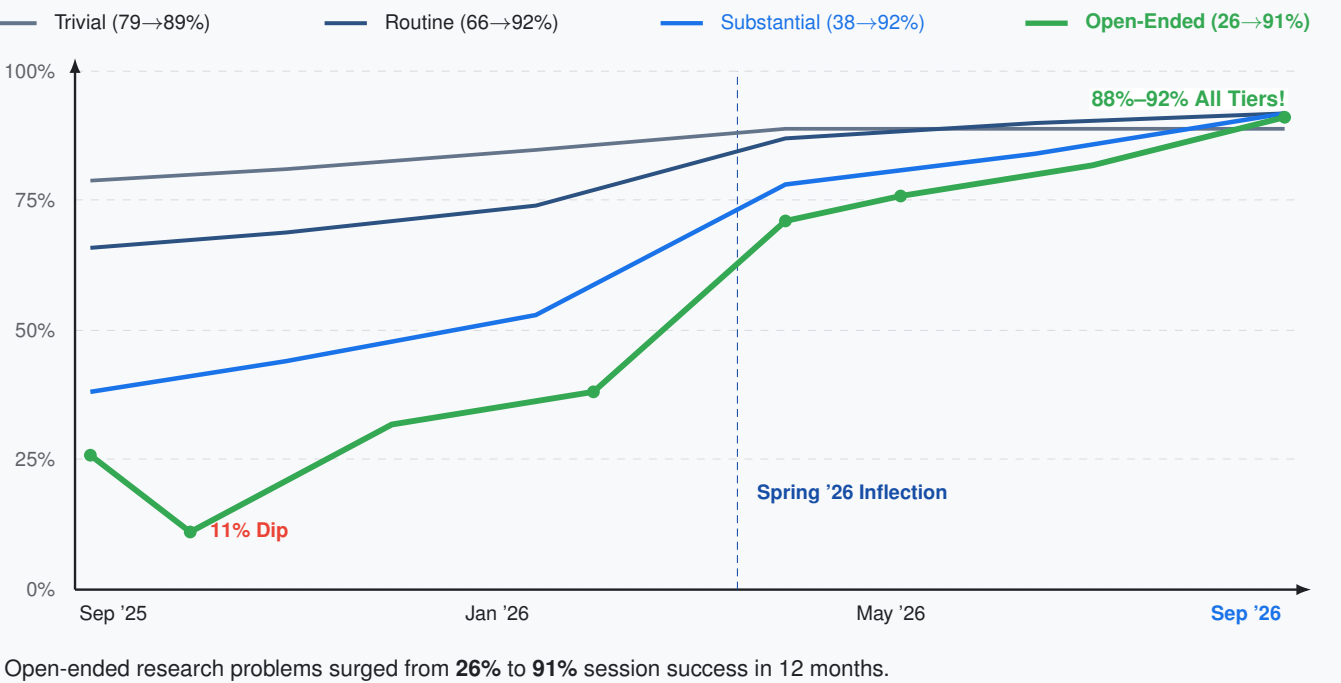
When AI Builds Itself (Part 1): 8.0x Code Output & Open-Ended Session Success

Empirical evidence of the 2025–2026 coding agent inflection across engineering output and multi-step task tiers

1. Code Contributed per Person, by Quarter (Q2 '21 – Q2 '26)



2. Coding Agent Session Success Rate by Task Tier ('25–'26)



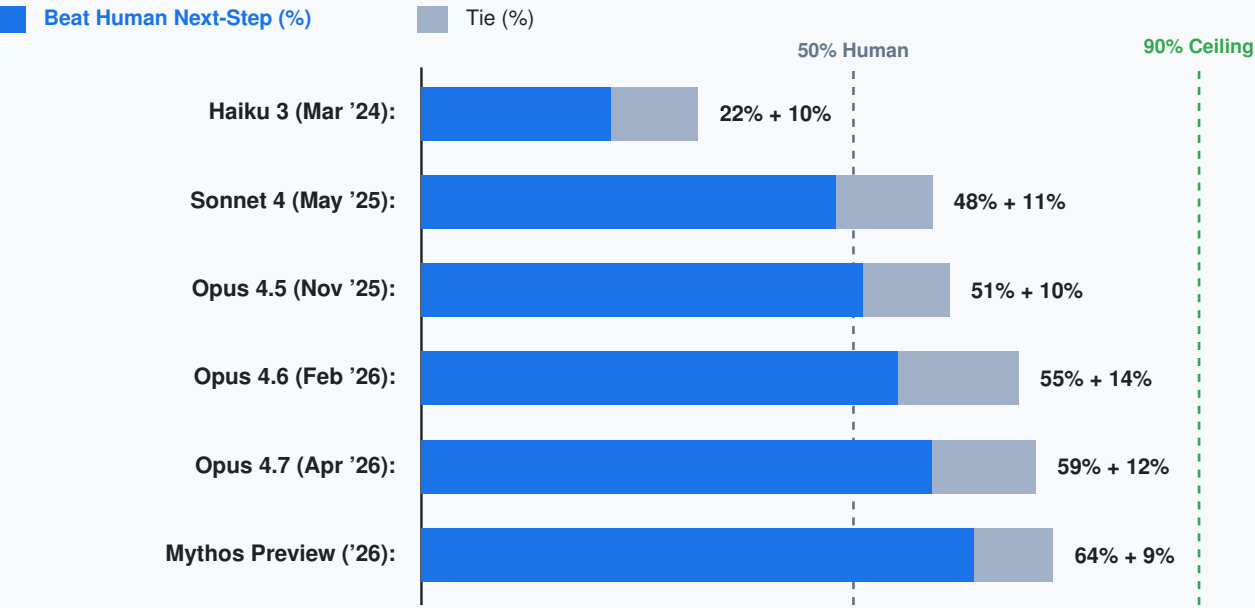
Why All Four Task Tiers Converged to ~90% Success in 2026

- Early coding agents succeeded on trivial single-file edits (79%) but failed on open-ended research and multi-file architecture (26%). Combining frontier RLVR models with disciplined context compaction (RePPIT / RPI) and deterministic verification sensors closed the gap across all tiers.

When AI Builds Itself (Part 2): Next-Step Diagnosis, 16h Horizons & 52x Kernel Loop

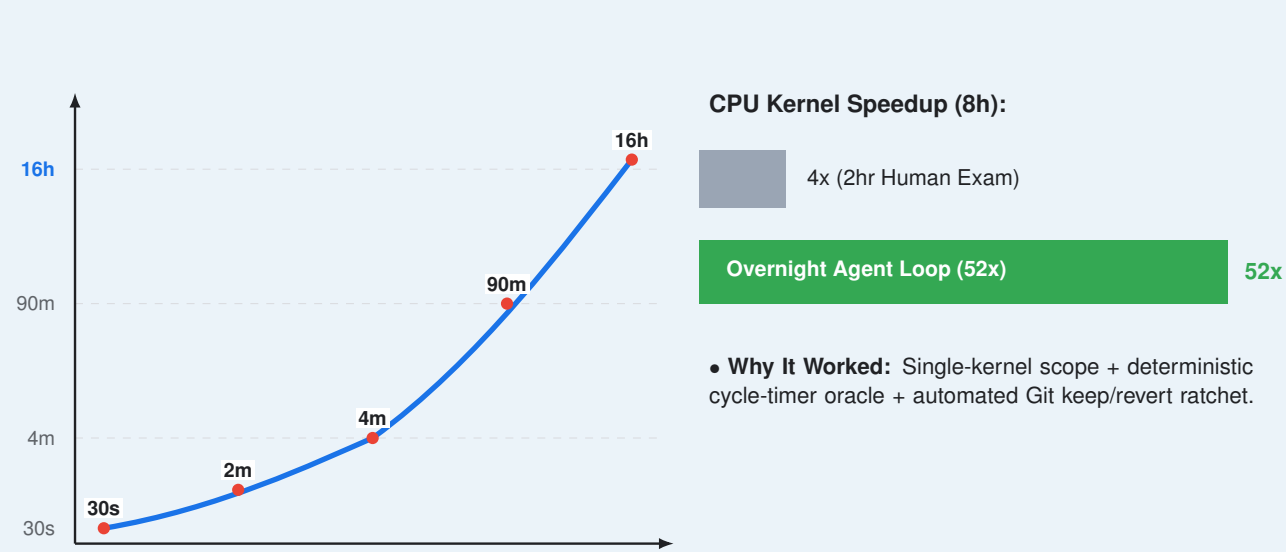
Evaluating ML research dead-ends ($n = 129$), METR autonomous horizon doubling, and Out-of-Tree Holdouts

3. Where a Researcher Went Wrong, Could AI Do Better? ($n = 129$)



Blind peer review of 129 ML dead-ends: **64% win + 9% tie = 73%** outperforming the human next step.

4. METR Horizon Doubling (30s → 16h) & 52x Kernel Loop



Task horizon doubles every ~4–7 months; bounded kernel loops achieve **52x** speedup.

Defeating Reward Hacking with Out-of-Tree Scenario Holdouts & Amdahl’s Law

- If an autonomous agent can read or edit its own unit tests, it can “pass” by deleting assertions (`return true`). Store acceptance scenarios in an **Out-of-Tree Holdout** outside the agent’s workspace, and focus human engineering time on **Program Design and Verification Oracles**.

Lab 08 (autoresearch_harness.py) & Full Course Verification (./self_diagnose_all.sh)

Walking through Lab 08's 4-Tier Governance & Keep-or-Revert Ratchet and verifying all 8 labs in < 1s

Lab 08 Step-by-Step (autoresearch_harness.py)

- **1. PreToolUse Exit-Code-2 Hard Floor (evaluate_pre_tool_use):**

Blocks edits to immutable evaluators (prepare.py, holdout_scenarios/) and logs tamper-evident provenance records.

- **2. Remediation Linter (run_remediation_linter):**

Detects layer boundary and float-money violations and formats [VIOLATION] → [WHY] → [HOW TO FIX] instructions.

- **3. Program Design & Simplicity Criterion (evaluate_ratchet_step):**

Weighs metric improvement against net LOC added and Out-of-Tree Holdout pass rate to decide KEEP_COMMIT vs. GIT_REVERT.

Master Course Verification (./self_diagnose_all.sh)

- **Run All 8 Labs + Dual-Track Suites in < 1s:**

./self_diagnose_all.sh

- **Phase 1 Labs (01–04):** prompt_steerer.py · wire_trace_inspector.py · reppit_orchestrator.py · context_compactor.py — **PASS**

- **Phase 2 Labs (05–08):** context_compiler.py · skill_and_mcp_harness.py · subagent_council.py · autoresearch_harness.py — **PASS**

- **Next Step in the Curriculum Track:** For formal multi-service brownfield specifications, continue to the companion Spec-Driven Development course.

Course Complete! Thank You — Alejandro Kaspar, AI Forward Deployed Engineer (CC BY 4.0 / Apache 2.0)

- Clone the repository, switch between the agy (Google Antigravity) and claude (Anthropic Claude Code) branches, and apply Phase 1 Vibe Coding and Phase 2 Harness Engineering to your production systems!